# IS1201 - Programming and Problem Solving

## Introduction to Arrays and Strings in C

# Can a Variable Hold More than One Value?

- So far we have used many types of variables, but each variable stores one value at a time:
  - One **int** or one **float** or one **char**

- Here's an example of an int variable, mark, that can hold any value - 56, 78, 90, 82
  - Since there are only four possible values, it isn't too difficult to change the variable's value manually

```
int mark = 56;
mark = 78;
mark = 90;
mark = 82;
```

# Number of Variables Required

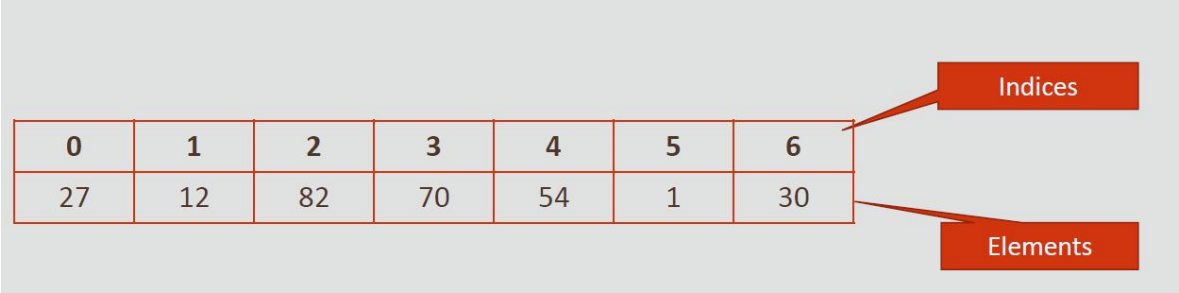
- But there are times when you'll need to hold more than one value in a variable
- What if you wanted to set aside a variable for each student's mark in a subject for a class of 300 students? That would mean 300 variables just for the marks in that subject!
- However, it can be time-consuming and tedious to create hundreds of variables

```
int mark1 = 56;
int mark2 = 78;
int mark3 = 90;
int mark4 = 82;
int mark5 = 52;
int mark6 = 75;
int mark7 = 43;
// .......
// .......
int mark300 = 54;
```

# Arrays Can Provide a Solution

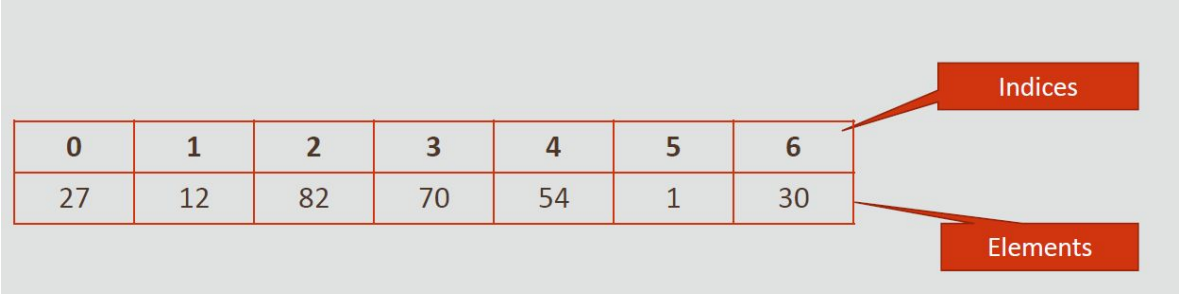
- ***An array in C is a collection of elements of the same data type, stored in contiguous memory locations, and accessed using a common name and an index.***

- In other terms:
  - An array is an indexed container that holds a set of values of a single type
  - Arrays allow you to create a single identifier to organize many items of the same date type



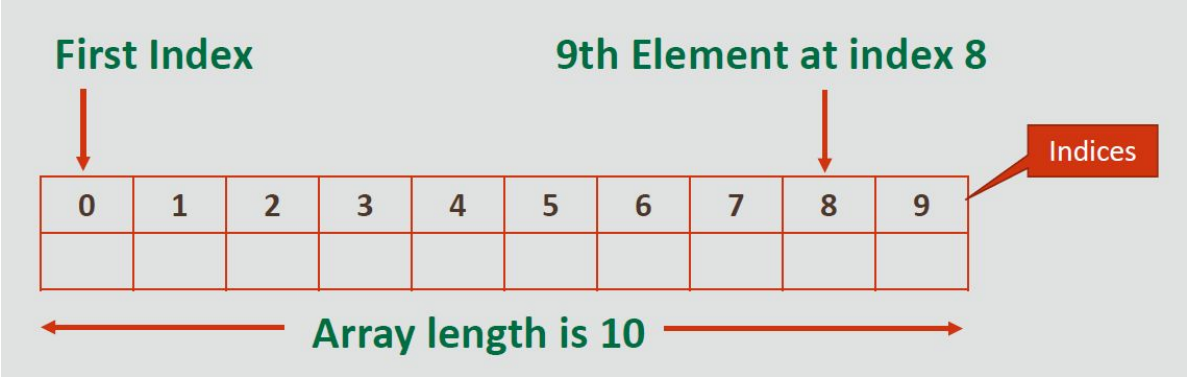
# Arrays Can Provide a Solution

- Each item in an array is called an element
- Arrays make storing and accessing a large number of values simple and easy



# Arrays Are Accessed by Their Index

- You can access each element in an array by its numerical index
- The index of the first element is 0
- A 10-element array has 0 to 9 indices



# Array Data Types

- Arrays can be of any data type, but all elements have to share the same type, such as: int, float, double or char

  - Array of int types

| | | | | | | |
|---|---|---|---|---|---|---|
| 27 | 12 | 82 | 70 | 54 | 1 | 30 |

# Declaring an Array

- Arrays, like all variables, must be declared prior to use
- You can declare an array by using the following syntax:

```
data_type array_name[array_size];
```

- *data_type → Type of elements (int, float, char, etc.)*
- *array_name → Identifier name for the array*
- *array_size → Number of elements the array can hold (must be a constant or fixed value)*

# Declaring an Array of Temperature Values

- Suppose you want to store 15 different temperature readings in an array

- You can declare an array as follows:

```
float temperatures[15];
```

Data type based on the items that you want to store in the array

Name of the array

Subscript

# Assigning Values to the Array

- What do the following code snippet do?

```
float temperatures[15];
temperatures[0] = 15;
temperatures[1] = 32;
temperatures[2] = 24;
temperatures[3] = 34;
temperatures[4] = 18
```

Variable Name

Index

Value

**What about the values stored in other indexes?**

# Important Note

- ***In C, memory spaces are not cleared from their previous value when variables or arrays are created. Because of this, it is generally good programming practice to not only declare your variables but to also initialize your arrays.***

# What About Declaring and Initializing an Array in a Single Step?

- You can also declare and initialize the array in a single step with known values:

```
data_type array_name[] = {value1, value2, value3, ...};
```

```
data_type array_name[array_size] = {value1, value2, value3, ...};
```

- You can quickly initialize your arrays with a single default value as demonstrated in the following array declaration

```
data_type array_name[array_size] = {0};
```

*Assigning the single numeric value of 0 in an array declaration will, by default, assign all array elements the value of 0.*

# Accessing Array Elements

- Arrays are sequential structures, meaning that items are stored one after another in an array
- You can access an individual element of an array by using a bracket notation
- For example, here's how you get values from the ages array:

```
int ages[5] = {12,34,16,56,70};

int myAge = ages[1];
int yourAge = ages[0];

printf("My age is %d. \n", ages[1]);
```

# How Do You Set the Value of an Array Element?

- You can set values to the array's elements like this:

```
int ages[5] = {12,34,16,56,70};

ages[0] = 56;
ages[4] = 54;
```

- After you set the values to the elements at indices 0 and 4, the ages array looks like this:

| **0** | 1 | 2 | 3 | **4** |
|---|---|---|---|---|
| 56 | 34 | 16 | 56 | 54 |
| ages[0] | ages[1] | ages[2] | ages[3] | ages[4] |

# How Do You Access the Length of an Array?

- So far, you created an array with a certain number of elements
- After creation, you can't change the length of an array. They can't grow beyond this size
- You can access the size of any array by using the array's **sizeof** operator
  - **sizeof** operator returns the size of a type in bytes

```
int ages[] = {12,34,16,56,70};

printf("%d \n", sizeof(ages)); //output = 20
```

- I you want to find out how many elements an array has, you can use the following formula (which divides the size of the array by the size of the first element in the array):

```
int ages[] = {12,34,16,56,70};

printf("Size of the array : %d \n", sizeof(ages)/sizeof(ages[0])); //output = Size of the array : 5
```

# Traversing an Array

- To iterate through, or traverse, an array means to process through each element of the array by index number

- You can access each element of an array to …
  - Print the elements
  - Search for an element
  - Initialize or update the elements of an array with some value

# Using a for Loop to Traverse Arrays

- You can use a for loop to traverse arrays
- The for loop lets you easily iterate over a known range

```
// Method 1
for(int i = 0; i < 5; i++){
   printf("%d \n", ages[i]);
}

// Method 2
int arrayLength = sizeof(ages)/sizeof(ages[0]);

for(int i = 0; i < arrayLength; i++){
    printf("%d \n", ages[i]);
}
```

# Putting It All Together

- Let’s look at an example where you need to …
  - Enter the scores of 10 students by using scanf() function
  - Display the scores that you entered
  - Calculate the average of the scores that you entered

# Let’s Compute the Average Score

```
// for loop to get the user input and insert them in to the array
for(int i = 0; i < 10; i++){
   printf("Enter the mark of the student %d : ", i+1);
   scanf("%d", &marks[i]);
}

// for loop to display the marks in the array
for(int i = 0; i < 10; i++){
   printf("Mark of the student %d : %d \n", i+1, marks[i]);
}

// for loop to calculate the sum of the marks
for(int i = 0; i < 10; i++){
   sum += marks[i];
}

printf("Sum of the marks : %d \n", sum);
printf("Average of the marks : %f \n", sum / 10.0);
```

# Exercises

1. Write a program in C to read n number of values in an array and display them in reverse order
2. Write a program in C to copy the elements of one array into another array
3. Write a program in C to count the total number of duplicate elements in an array
4. Write a program in C to print all unique elements in an array
5. Write a program in C to find the largest and smallest numbers in an array
6. Write a program in C to sort an array in both ascending and descending order
7. Write a program in C to delete an element at a desired position from an array
8. Write a program in C to find the second largest element in an array
9. Write a program in C to insert only unique numbers into an array
10. Write a program in C to insert numbers into an array and check if it is a palindrome

# Multidimensional Arrays

- A multidimensional array is basically an array of arrays
- Arrays can have any number of dimensions

- Let's have a look at **two-dimensional** arrays
  - The easiest way to understand or think about two-dimensional arrays is to visualize a table with rows and columns (e.g. a checkerboard, chessboard, or spreadsheet)
  - C implements two-dimensional arrays as single-dimension arrays with pointers to other single-dimension arrays

# Two-Dimensional Arrays

- You can declare two-dimensional array using following syntax:

```
data_type array_name[rows][columns];
```

- *Data_type → The type of elements (int, float, char, etc.)*
- *Rows → Number of rows*
- *Columns → Number of columns*

# Initializing 2D Arrays

```
int arrayTwoD[3][3] = {{0,1,2},{0,1,2},{0,1,2}};
```

| | Column 0 | Column 1 | Column 2 |
|---|---|---|---|
| Row 0 | 0 | 1 | 2 |
| Row 1 | 0 | 1 | 2 |
| Row 2 | 0 | 1 | 2 |

# Access the Elements of a 2D Array

- To access a specific element in a two-dimensional array, use the following syntax:

```
array_name[row_index][column_index];
```

- *row_index → the row number (starting from 0)*
- *column_index → the column number (starting from 0)*

# Access the Elements of a 2D Array

```
int arrayTwoD[3][3] = {
   // {Column 0, Column 1, Column 2}
   {10,20,30}, // Row 0
   {20,40,60}, // Row 1
   {40,80,120} // Row 2
};

printf("%d \n", arrayTwoD[2][1]); // Prints 80
```

| | Column 0 | Column 1 | Column 2 |
|---|---|---|---|
| Row 0 | 10 | 20 | 30 |
| Row 1 | 20 | 40 | 60 |
| Row 2 | 40 | **80** | 120 |

# Change Elements in a 2D Array

```c
int arrayTwoD[3][3] = {
   // {Column 0, Column 1, Column 2}
   {10,20,30}, // Row 0
   {20,40,60}, // Row 1
   {40,80,120} // Row 2
};

printf("%d \n", arrayTwoD[2][1]); // Prints 80

// Assigning a new value for the element in the third row (2) and second column (1)
arrayTwoD[2] [1] = 0;

printf("%d \n", arrayTwoD[2][1]); // Prints 0
```

| | Column 0 | Column 1 | Column 2 |
|---|---|---|---|
| Row 0 | 10 | 20 | 30 |
| Row 1 | 20 | 40 | 60 |
| Row 2 | 40 | **80** | 120 |

| | Column 0 | Column 1 | Column 2 |
|---|---|---|---|
| Row 0 | 10 | 20 | 30 |
| Row 1 | 20 | 40 | 60 |
| Row 2 | 40 | **0** | 120 |

# Traverse Through a 2D Array

- To loop through a multidimensional array, you need one loop for each of the array's dimensions

Example:

```
int arrayTwoD[3][3] = {
   // {Column 0, Column 1, Column 2}
   {10,20,30}, // Row 0
   {20,40,60}, // Row 1
   {40,80,120} // Row 2
};

for(int i = 0; i < 3; i++){
   for (int j = 0; j < 3; j++){
      printf("Value of the element in the row - %d and column - %d is %d \n", i, j, arrayTwoD[i][j]);
   }
}
```

Output:

```
Value of the element in the row - 0 and column - 0 is 10
Value of the element in the row - 0 and column - 1 is 20
Value of the element in the row - 0 and column - 2 is 30
Value of the element in the row - 1 and column - 0 is 20
Value of the element in the row - 1 and column - 1 is 40
Value of the element in the row - 1 and column - 2 is 60
Value of the element in the row - 2 and column - 0 is 40
Value of the element in the row - 2 and column - 1 is 80
Value of the element in the row - 2 and column - 2 is 120
```

# Exercises

1. Write a program in C to input values into a 2D array and print them in matrix form
2. Write a program in C to calculate and display the sum of all elements in a 2D array
3. Write a program in C to calculate and display the sum of each row and each column
4. Write a program in C to add two matrices and display the result
5. Write a program in C to scan a matrix and find the largest and smallest values

# Strings

- We've used strings before

```
printf("Hello, World");
```

String Literal

- Strings are used for storing text/characters
- Strings are arrays with base type char
  - One character per element
  - One extra character: '\0'
    - Called 'null character'
    - End marker

Null Character

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| H | e | l | l | o | , | | W | o | r | l | d | \0 |

# Declaring a String

- Strings can be declared as an array of characters

```
char name[10];
```

- Above example declares a char array variable to hold up to 9 characters plus one null character

- Typically strings are partially filled arrays
  - Declare large enough to hold max-size string, including the null character

# Initializing a String

- String can be initialized as follows:

```
char name[10] = "John Doe";
```

- Places '\0' at the end

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| J | o | h | n | | D | o | e | \0 | ? |

- Another way:

```
char name[] = {'J','o','h','n','\0'};
```

# String Indexes

```
char name[]= "John Doe";
```

| J | o | h | n | | D | o | e | \0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

Indexes

- Strings also act as arrays
- Strings can be manipulated by changing elements

```
char name[]= "John Doe";

// Prints name before changing an element
printf("%s \n",name);

// Prints the name after chaning an element
name[3] = 'k';
printf("%s \n",name);
```

- If you are manipulating the elements of a string, make sure not to modify the null character ('\0'), as it may lead to unexpected or erroneous behavior

# Special Characters/ Escape Sequences

- Special characters in C are non-printable or control characters used for formatting, control, or special syntax. They are written using escape sequences
- Consider the following example:

```
char quote[] = "He said, "Hello, World!" with a smile.";
```

- The solution to avoid this problem, is to use the backslash escape character
- The backslash (\) escape character turns special characters into string characters

```
char quote[] = "He said, \"Hello, World!\" with a smile.";
```

- The sequence \"  inserts a double quote in a the string

# When to Use Escape Sequences?

- To include characters like quotes inside strings
- To format output (new lines, tabs, etc.)
- To control behavior (e.g., alert sound)

Examples:

Code:

```
printf("Name\tAge\nJohn\t25\nJane\t30");
```

Output:

```
Name    Age
John    25
Jane    30
```

# Common Escape Sequences

| Escape Sequence | Meaning |
|---|---|
| \n | New line |
| \t | Horizontal tab |
| \\ | Backslash (\) |
| \" | Double quote (") |
| \' | Single quote (') |
| \r | Carriage return |
| \b | Backspace |
| \a | Alert (bell sound) |
| \0 | Null character |

# String Functions

- C has many useful string functions, which can be used to perform different operations on strings
- To use them, you must include the <string.h> header file in your program

```
#include <string.h>
```

Example:

```
#include <stdio.h>
#include <string.h>    // String library

int main() {
    // Using strlen() to find the length of the string "SRI LANKA"
    // strlen returns the number of characters excluding the null terminator '\0'
    printf("Length of the string \"SRI LANKA\" is %d", strlen("SRI LANKA"));

    return 0;
}
```

Output:

```
Length of the string "SRI LANKA" is 9
```

# Common String Functions

| Function | Description |
|---|---|
| strlen() | Get string length |
| strcpy() | Copy one string to another |
| strcat() | Concatenate two strings |
| strcmp() | Compare two strings |
| strrev() | Reverse a string (non-standard) |
| strchr() | Find first occurrence of a character |
| strstr() | Find substring in a string |

# Length & Copying

- **`strlen()`**

  Returns the number of characters in a string excluding the null terminator

  Example:
  ```
  char name[] = "John";
  int len = strlen(name);
  ```

  **Note:** `sizeof` returns the allocated memory (including the null byte), whereas `strlen` counts visible characters

- **`strcpy()` & `strncpy()`**

  `strcpy` copies an entire source string into a destination buffer (including the null terminator)

  `strncpy` copies at most n characters and pads with a null byte if the source is shorter

  ```
  char name[] = "Hello World!";
  strncpy(name,"Johny Doe",5);
  printf("%s",name); // Outputs Johny World!
  ```

# Concatenation

- **`strcat()`**

  Appends the source string to the destination, replacing the destination's null terminator

  Destination must be large enough

- **`strncat()`**

  Appends at most n characters from source and always null terminates the result

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char lastName[] = "Doe";
    char name[10] = "John ";
    strcat(name,lastName);
    printf("%s", name); // outputs John Doe

}
```

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char text[] = "Doe is a teenager.";
    char name[10] = "John ";
    strncat(name,text,3);
    printf("%s", name); // outputs John Doe

}
```

# Comparison

- **`strcmp()`**

  Lexicographically compares two strings

- **`strncmp()`**

  Compares at most n characters of two strings using the same return convention

  Example: Consider there are two strings S1 and S2 and you want compare it

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char S1[] = "John";
    char S2[] = "John";

    int output = strcmp(S1,S2);
    printf("%d", output); // outputs 0

}
```

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char S1[] = "John";
    char S2[] = "Doe";

    int output = strcmp(S1,S2);
    printf("%d", output); // outputs 1

}
```

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char S1[] = "John";
    char S2[] = "Zebra";

    int output = strcmp(S1,S2);
    printf("%d", output); // outputs -1

}
```

# Comparison

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char S1[] = "cat";
    char S2[] = "car";

    int output = strncmp(S1,S2,2);
    printf("%d", output); // outputs 0

}
```

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char S1[] = "cat";
    char S2[] = "car";

    int output = strncmp(S1,S2,3);
    printf("%d", output); // outputs 1

}
```

```c
#include <stdio.h>
#include <string.h>    // String library

int main() {

    char S1[] = "cab";
    char S2[] = "cat";

    int output = strncmp(S1,S2,3);
    printf("Output : %d", output); // outputs -1

    return 0;

}
```

| Return | Meaning |
|---|---|
| < 0 | S1 is lexicographically smaller |
| 0 | Strings are equal |
| > 0 | S1 is lexicographically greater |

# Searching

- **`strchr()`**

  Finds the first occurrence of a character in a string; returns pointer or NULL

- **`strrchr()`**

  Finds the last occurrence of a character in a string; returns pointer or NULL

- **`strstr()`**

  Searches for the first occurrence of a substring in another string; return pointer or NULL

# Searching

Examples:

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "The quick brown fox jumps over the lazy dog";
    char letter = 'b';

    char *found = strchr(str, letter);
    if (found != NULL)
        printf("Found '%c' at position: %ld\n", letter, found - str);
    else
        printf("'%c' not found.\n", letter);

    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "The quick brown fox jumps over the lazy dog";
    char word[] = "brown";

    char *found = strstr(str, word);
    if (found != NULL)
        printf("Found '%s' at position: %ld\n", word, found - str);
    else
        printf("'%s' not found.\n", word);

    return 0;
}
```

# Formatting

- **`sprintf()`**

  Formats data into a string buffer rather than printing to stdout

  Useful for constructing messages

  Example:

```c
#include <stdio.h>

int main() {
    char buffer[100];
    char name[] = "Alice";
    int age = 25;
    float score = 89.75;

    // Format the string and store it in 'buffer'
    sprintf(buffer, "Name: %s, Age: %d, Grade: %.2f", name, age, score);

    // Print the formatted string
    printf("%s\n", buffer);

    return 0;
}
```

Output:

```
Name: Alice, Age: 25, Grade: 89.75
```

# Tokenizing

- **`strtok()`**

  Split the string into tokens based on the delimiters

  Modifies the input string by replacing delimiters with null bytes

Example:

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "C is a powerful programming language";
    char *token;

    // strtok modifies the original string, so we use a copy if needed
    token = strtok(str, " ");  // split by space

    while (token != NULL) {
        printf("Token: %s\n", token);
        token = strtok(NULL, " ");  // NULL continues from last position
    }

    return 0;
}
```

Output:

```
Token: C
Token: is
Token: a
Token: powerful
Token: programming
Token: language
```

# Any Questions?

- Post them in the discussion forum : Click here
- Send an email to : rns@ucsc.cmb.ac.lk